# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY**
**COURSE CODE: CSA51**

**COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

**Debugging & Optimisation Task : 10%**

**QUESTIONS:** **Total Marks: 50**

**Annexure A**

***Code of Conduct:***

*I* **L.dattu kumar(192324141)(***Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** **L.Dattu Kumar**

**Annexure A**
**Debugging & Optimisation Task**

## Question 4: Optimization and Security Analysis of Diffie–Hellman Key Exchange

### A. Debug incorrect shared-key generation due to improper modular arithmetic

Problem identification

Diffie–Hellman is a public-key cryptographic technique that allows two users to establish a common secret key over an insecure network without directly transmitting the secret key.

A Python implementation may generate incorrect shared keys due to improper modular arithmetic, incorrect public parameters, invalid public values or incorrect exponentiation.

Working principle

Let:

- \(p\) = a large prime number.
- \(g\) = a generator.
- \(a\) = Alice's private key.
- \(b\) = Bob's private key.

Alice calculates her public key:

\[
A=g^a\bmod p
\]

Bob calculates his public key:

\[
B=g^b\bmod p
\]

They exchange their public keys. Alice calculates:

\[
K_A=B^a\bmod p
\]

Bob calculates:

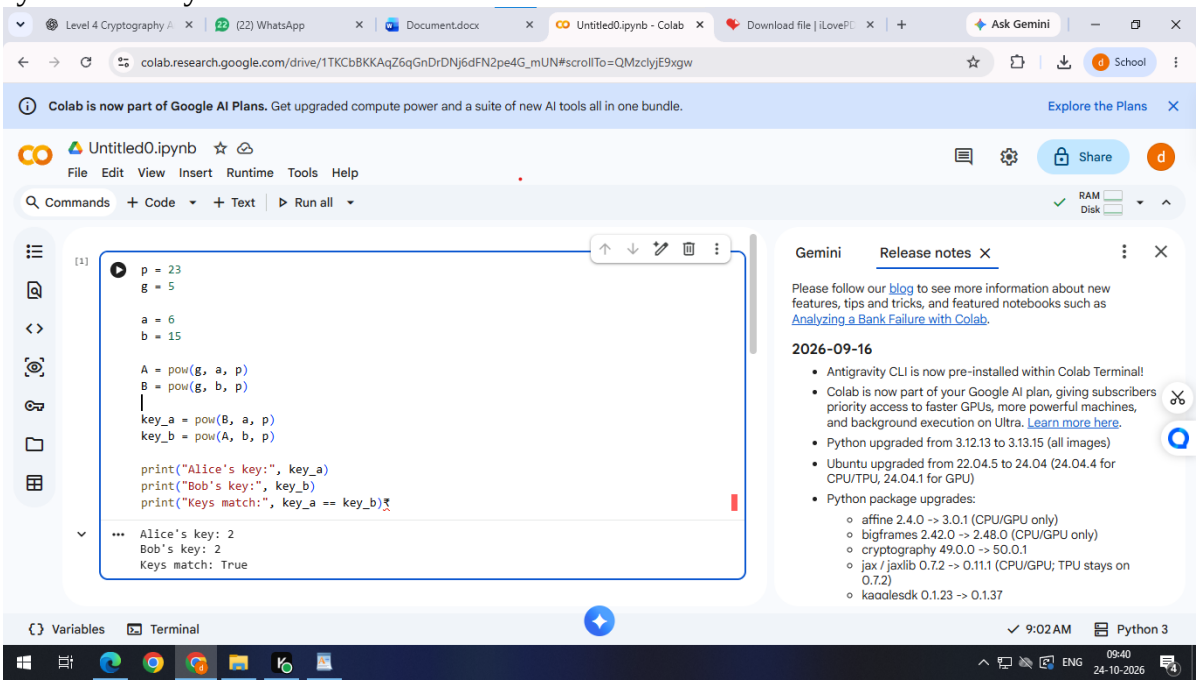
\[
K_B=A^b\bmod p
\]

Both should obtain the same shared secret because:

\[

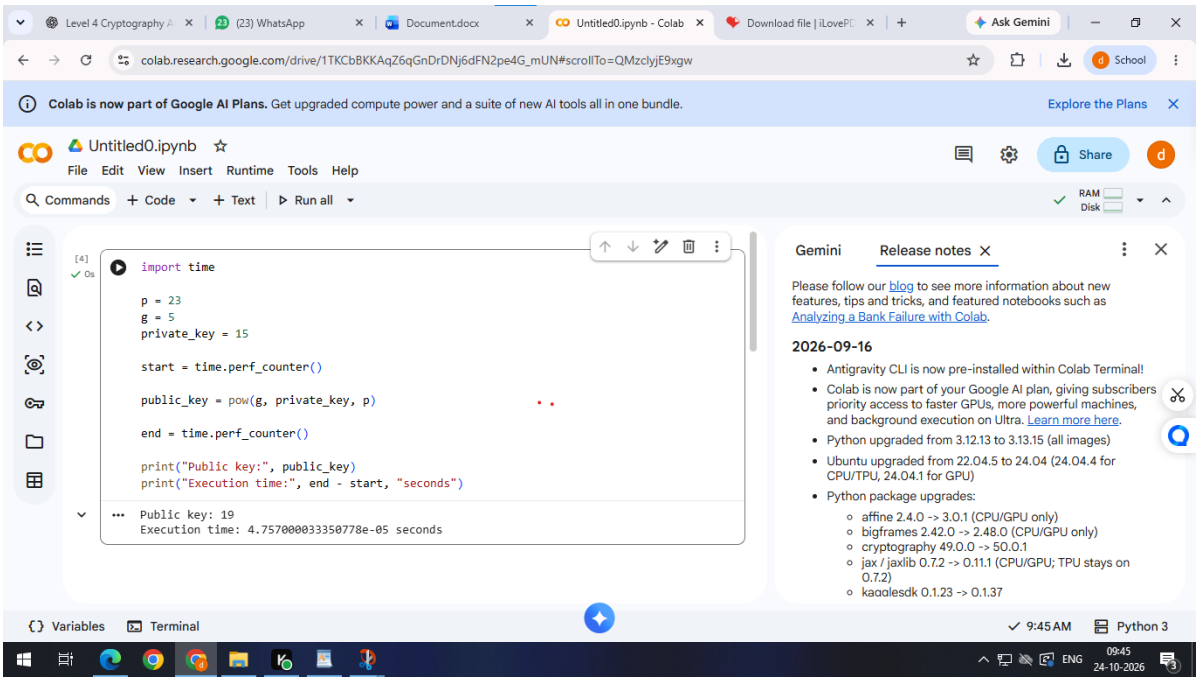
K_A=K_B=g^{ab}\bmod p
\]

Errors and debugging solutions
Error Solution
Incorrect prime or generator Use validated cryptographic parameters.
Incorrect modular exponentiation Use Python's pow(base, exponent, modulus).
Invalid received public key Validate the key according to the selected group or protocol.
Private key accidentally exchanged Exchange only public keys.
Shared keys do not match Check parameters, exchanged values and calculations systematically.




Level 4 debugging method: Test the implementation with known small demonstration values, verify each public-key calculation independently, compare both shared secrets, and then test with approved production parameters using a cryptographic library.

**B. Optimize exponentiation operations for large prime values**
Problem
Calculating large powers directly, such as \(g^a\), creates very large intermediate numbers. This wastes memory and processing time.
Optimization technique
Python's built-in pow() function performs modular exponentiation efficiently.



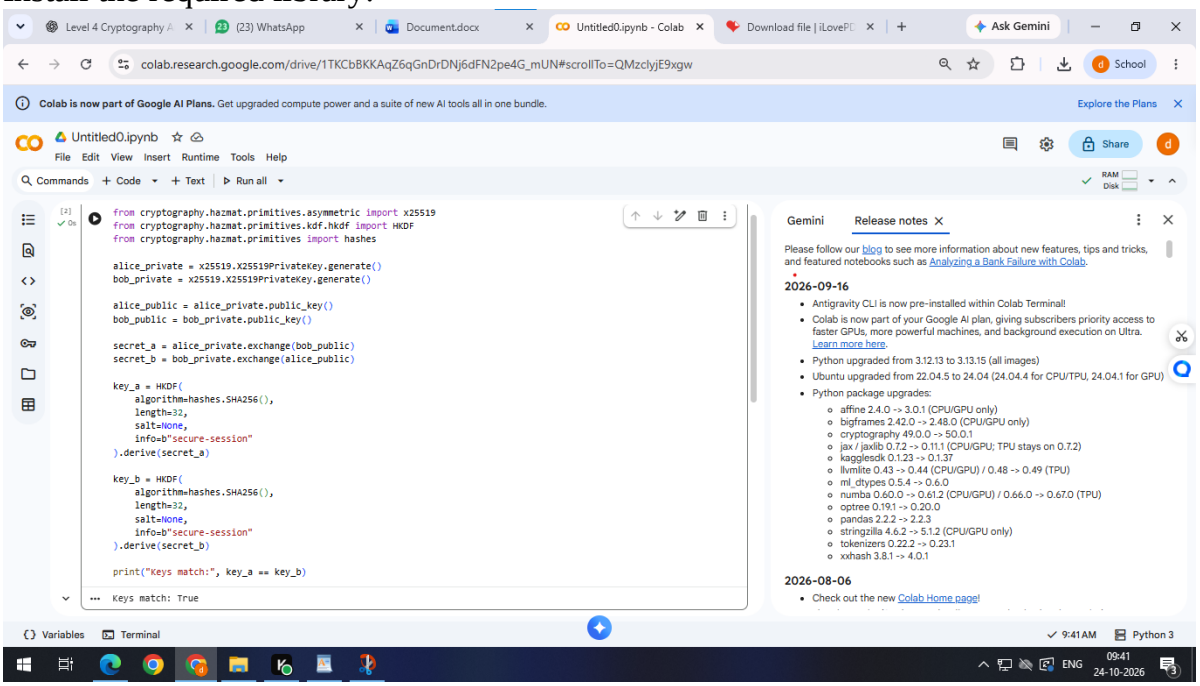

Explanation
1. Alice and Bob choose their private values.
2. Each calculates a public key using modular exponentiation.
3. They exchange public keys without revealing their private values.
4. Each calculates the shared secret independently.
5. The program verifies that both shared secrets match.
The three-argument pow() function avoids constructing the full intermediate power and is more efficient than pow(g, a) % p.

Performance improvement: Efficient modular exponentiation reduces computational overhead, especially for large exponents. For production systems, standardized groups and established cryptographic libraries should be used. Small demonstration parameters such as \(p=23\) are not secure.

**C. Prevent man-in-the-middle attacks and evaluate performance**
Problem identification
A man-in-the-middle (MITM) attack occurs when an attacker intercepts communication between Alice and Bob and replaces their public keys with the attacker's own keys.

Ordinary Diffie–Hellman establishes a shared secret, but it does not authenticate the identities of the communicating users. Therefore, an attacker may establish separate shared secrets with Alice and Bob and relay messages between them.
Security enhancements
1. Authentication: Use digital certificates or digital signatures to verify identities.
2. Secure key exchange: Use an authenticated protocol such as TLS.

3. Ephemeral keys: Generate fresh private values for each session to support forward secrecy when used in an appropriate protocol.
4. Key derivation: Use HKDF to derive suitable session keys from the shared secret.
5. Public-key validation: Validate received public keys according to the selected cryptographic protocol.
Python implementation using X25519 and HKDF
Install the required library:




Performance and security analysis
X25519 provides efficient elliptic-curve key agreement with relatively small public keys. HKDF converts the shared secret into a suitable session key. Authentication adds processing and communication overhead, but it is necessary to prevent impersonation.

This example demonstrates key agreement and key derivation; it does not implement authentication. A production application must use a properly authenticated protocol.

Conclusion: Efficient modular exponentiation or standardized elliptic-curve key agreement improves performance. Authentication and secure key derivation strengthen security. These improvements make the key-exchange process more efficient and resistant to attacks.

**Question 5: Debugging and Enhancing Elliptic Curve Cryptography (ECC)**

**A. Identify and fix errors in point addition and scalar multiplication**

Problem identification

Elliptic Curve Cryptography (ECC) uses mathematical operations on points on an elliptic curve to provide cryptographic functions such as key exchange and digital signatures.

A common elliptic-curve equation is:

$$y^2=x^3+ax+b\pmod p$$

An ECC program may produce incorrect results when point addition, point doubling or scalar multiplication is implemented incorrectly.

Common errors and solutions

| Error | Corrective action |
|---|---|
| Incorrect slope calculation | Apply the correct point-addition formula. |
| Incorrect modular inverse | Calculate the inverse modulo $p$, not ordinary division. |
| Point doubling not handled | Use the separate point-doubling formula. |
| Point at infinity ignored | Implement the identity element correctly. |
| Invalid curve points accepted | Verify points against the curve equation and applicable subgroup requirements. |
| Incorrect scalar multiplication | Test the double-and-add algorithm against known results. |

Correct mathematical formulas

For two distinct points $P=(x_1,y_1)$ and $Q=(x_2,y_2)$, calculate the slope:

$$\lambda=(y_2-y_1)(x_2-x_1)^{-1}\pmod p$$

For point doubling, calculate:

$$\lambda=(3x_1^2+a)(2y_1)^{-1}\pmod p$$

Then calculate the resulting point $R=(x_3,y_3)$:

$$x_3=\lambda^2-x_1-x_2\pmod p$$

$$y_3=\lambda(x_1-x_3)-y_1\pmod p$$

\]

Here, the inverse notation means a modular multiplicative inverse. Special cases must be handled separately, including point doubling when the inverse does not exist and adding a point to its inverse.
Systematic debugging procedure
1. Check that the selected curve parameters are valid.
2. Verify that the input points satisfy the curve equation.
3. Test point addition using known valid points.
4. Test point doubling separately.
5. Verify the identity and inverse-point cases.
6. Compare scalar multiplication results against a trusted implementation.
Conclusion: Correct modular arithmetic, exceptional-case handling and systematic testing eliminate common ECC implementation errors and improve correctness.

**B. Optimize ECC using the double-and-add algorithm**
Problem
Scalar multiplication calculates:
\[
Q=kP
\]

where \(k\) is a scalar and \(P\) is a point on the curve.

Repeatedly adding \(P\) to itself \(k\) times is inefficient for large values of \(k\).
Optimization technique: Double-and-add
The double-and-add algorithm uses the binary representation of the scalar to reduce the number of point operations.

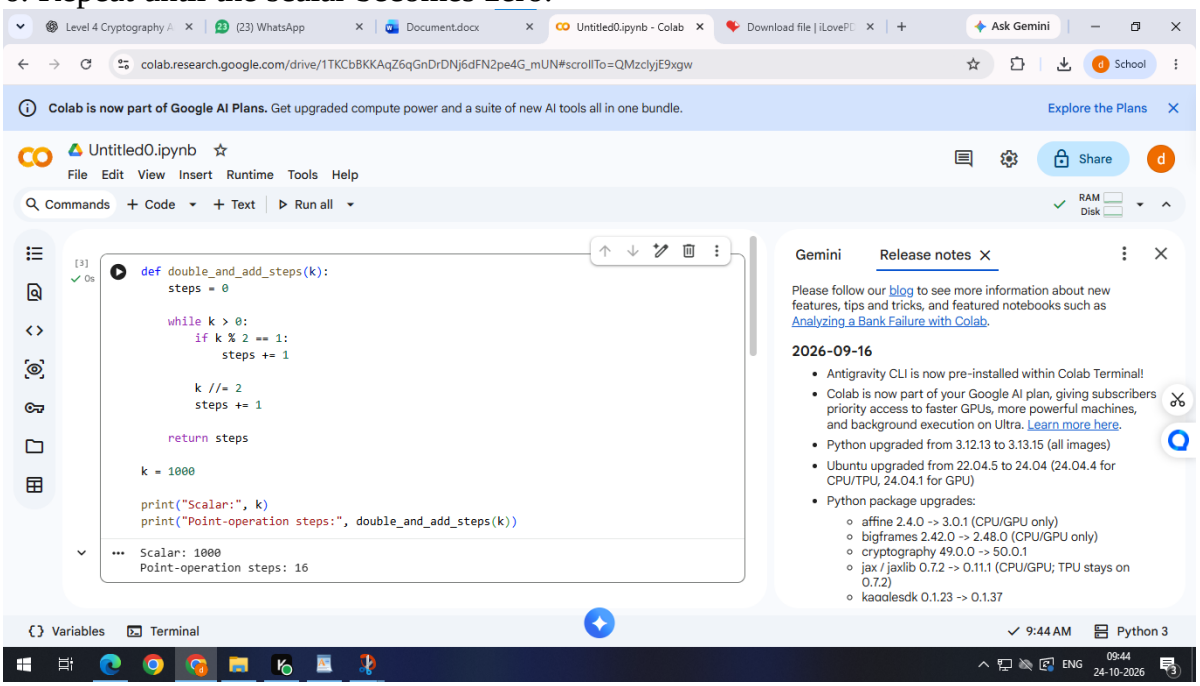
Algorithm:
result = Point at Infinity
current = P

while k > 0:
    if k is odd:
        result = result + current

    current = current + current
    k = k // 2

return result

Explanation
1. Initialize the result to the point at infinity.
2. Check whether the scalar is odd.
3. If it is odd, add the current point to the result.
4. Double the current point.
5. Divide the scalar by two using integer division.
6. Repeat until the scalar becomes zero.




Performance analysis
- Repeated addition requires \(O(k)\) point-operation steps.
- Double-and-add requires \(O(\log k)\) point-operation steps.
- This reduces the number of operations substantially for large scalars.
However, the basic algorithm above is pseudocode, not a complete ECC implementation. A straightforward implementation may leak information through execution time or other side channels. Production implementations should use tested libraries and constant-time techniques where required.

Conclusion: Double-and-add improves computational efficiency by reducing point operations while preserving the mathematical result. Secure implementation practices are still necessary to protect private keys.

**C. Compare optimized ECC with RSA and justify its suitability for resource-constrained environments**
Comparison
Feature ECC RSA
Key size Smaller Larger

Security Strong with suitable curves Strong with suitable key sizes
Computational cost Efficient for many key-agreement operations Private-key operations can be computationally expensive
Memory requirements Generally lower Generally higher
Communication overhead Lower Higher
Suitable applications Mobile devices, IoT and embedded systems
Compatibility-focused systems and established infrastructure

For comparable classical security, a properly implemented 256-bit elliptic-curve system provides approximately 128-bit security, similar to RSA with a 3072-bit key.
Analysis
1. Key size: ECC uses smaller keys than RSA for comparable classical security. This reduces storage requirements.

2. Processing: ECC can perform key agreement efficiently on devices with limited processing power. Actual performance depends on the algorithm, implementation and hardware.

3. Communication: Smaller public keys reduce the amount of data transmitted over networks.

4. Security: Both ECC and RSA can provide strong security when suitable parameters and secure implementations are used.

5. Resource-constrained environments: ECC is often advantageous for IoT devices, embedded systems and mobile applications because it reduces key storage and communication overhead.
Conclusion
ECC is generally suitable for resource-constrained environments because it provides strong classical security with smaller keys and lower communication overhead. Standardized algorithms such as X25519 should be used for key agreement, while approved elliptic-curve signature schemes should be used for authentication. Custom ECC implementations should be avoided in production systems.

**RUBRICS FOR CALCULATION**

| **Criteria** | **Weightage** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** |
|---|---|---|---|---|---|
| Problem | | Accurately identifies all errors and | Identifies most errors | Identifies some errors | Unable to identify |

| | | | | | |
|---|---|---|---|---|---|
| Identificati on | 20% | root causes with clear understandin g | with minor gaps | but misses key issues | errors correctly |
| Debugging | 25% | Applies systematic debugging techniques effectively to resolve issues | Uses appropriate debugging methods with minor errors | Limited debugging approach; requires guidance | Unable to debug or resolve issues |
| Optimizati on | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity | No improvemen t; inefficient approach retained |
| Analysis | 20% | Demonstrate s strong logical reasoning and clear justification of solutions | Shows reasonable analysis with some justification | Limited analysis; weak reasoning | No analytical reasoning demonstrate d |
| Code Quality | 15% | Produces clean, well-structured code with proper comments and documentatio n | Code is mostly clear with minor documentatio n gaps | Code lacks structure and proper documentatio n | Poorly written code with no documentati on |